

Ejemplo: Queremos enviar el mensaje **0110**

- 1) Determinar cuántos bits de paridad hay que incluir teniendo en cuenta el tamaño del mensaje que queremos enviar.

Hay que añadir **p** de bits de paridad para un mensaje de **d** bits teniendo en cuenta la siguiente relación

$$2^p \geq d + p + 1$$

En este caso:

- $d = 4$

Para determinar **p** podemos resolver la inecuación sustituyendo **d** y probando varios valores de **p** hasta encontrar el primero que cumpla la expresión

$$2^p \geq 4 + p + 1$$

- $p = 1$

$$2^1 \geq 4 + 1 + 1$$
$$2^1 \geq 6$$

Cómo hemos llegado a una expresión falsa (dado que 2 no es mayor o igual a 6), deducimos que **p** tiene que ser un número más grande, con lo cual seguimos probando números

- $p = 2$

$$2^2 \geq 7$$

Sigue siendo falso.

- $p = 3$

$$2^3 \geq 8$$

Esta vez sí que se cumple la expresión dado que 2 elevado a 3 es 8. Por lo tanto, necesitamos tres bits de paridad. En total necesitamos 7 bits (3 de paridad y 4 de datos) para enviar el mensaje.

- 2) Crear una tabla donde todos los bits del código de humming son las columnas y numerar cada columna empezando por el 1. Pondremos también el número de la columna expresado en binario.

bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1
111	110	101	110	011	010	001

- 3) Marcaremos como bits de paridad aquellos cuyo número en binario tenga únicamente un 1, o lo que es lo mismo, que sea una potencia de dos (1, 2, 4, 8, 16 ...). El resto de bits serán los bits de datos.

d4	d3	d2	p3	d1	p2	p1
bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1
111	110	101	100	011	010	001

- 4) Añadimos tantas filas como bits de paridad haya. El objetivo es saber qué bit de paridad se encarga de qué grupo de bits de datos.

Para ello, cogemos el número en binario del bit de paridad en cuestión y hacemos la operación AND bit a bit con cada columna. Si el resultado tiene algún 1, ese bit de paridad se encarga del bit de datos en cuestión

	d4	d3	d2	p3	d1	p2	p1
	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1
	111	110	101	100	011	010	001
p1							
p2							
p3							

Comenzamos con el bit de paridad p1 (cuyo número en binario es 001). Para saber de qué bits se preocupa hay que hacer la operación AND bit a bit entre 001 y el resto de números.

```

    001
AND 010
-----
    000

```

El resultado es 000 $\Rightarrow$ el bit de paridad p1 no se encarga del bit p2

```

    001
AND 011
-----
    001

```

Ahora el resultado tiene al menos un uno $\Rightarrow$ el bit de paridad p1 se encarga del bit d3

Continuamos el proceso y señalamos en la tabla dichos casos. Esta operación se puede realizar mentalmente muy rápido comprobando que ambas cadenas binarias coincidan con un 1 en la misma posición, en cuyo caso el resultado no puede ser 000

	d4	d3	d2	p3	d1	p2	p1
	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1
	111	110	101	100	011	010	001
p1							
p2							
p3							

De la tabla extraemos la siguiente asociación:

- p1 se encarga de controlar los bits de datos d1, d2 y d4
- p2 se encarga de controlar los bits de datos d1, d3 y d4
- p3 se encarga de controlar los bits de datos d2, d3 y d4

- 5) Finalmente toca formar el mensaje final. Primero metemos en cada bit de datos los bits del mensaje original (queremos enviar el mensaje **0110**) dejando en blanco los bits de paridad.

d4	d3	d2	p3	d1	p2	p1
0	1	1		0		

- 6) Ahora simplemente falta determinar el valor de los bits de paridad. Para ello tenemos que tener la paridad del subconjunto de cada bit de paridad.

Sabemos gracias al paso 4 que el bit de paridad p1 se encarga de los bits d1, d2 y d4.

$$d1 = 0$$

$$d2 = 1$$

$$d4 = 0$$

Como hay un número impar de unos (solo hay uno), el bit de paridad p1 vale 1

El bit de paridad p2 se encarga de los bits d1, d3 y d4

$$d1 = 0$$

$$d3 = 1$$

$$d4 = 0$$

$$\Rightarrow p2 = 1 \text{ otra vez}$$

Finalmente, el bit de paridad p3 se encarga de d2, d3 y d4

$$d2 = 1$$

$$d3 = 1$$

$$d4 = 0$$

$$\Rightarrow p3 = 0 \text{ dado ahora tenemos un número par de 1's}$$

d4	d3	d2	p3	d1	p2	p1
0	1	1	0	0	1	1

$\Rightarrow$ El mensaje a enviar sería **0110011**